Auto Scaling Web Architecture using EC2 and Application Load Balancer

AWS Compute Architecture Project

Demonstrating scalable and self-healing cloud infrastructure using EC2 Auto Scaling and Application Load Balancer

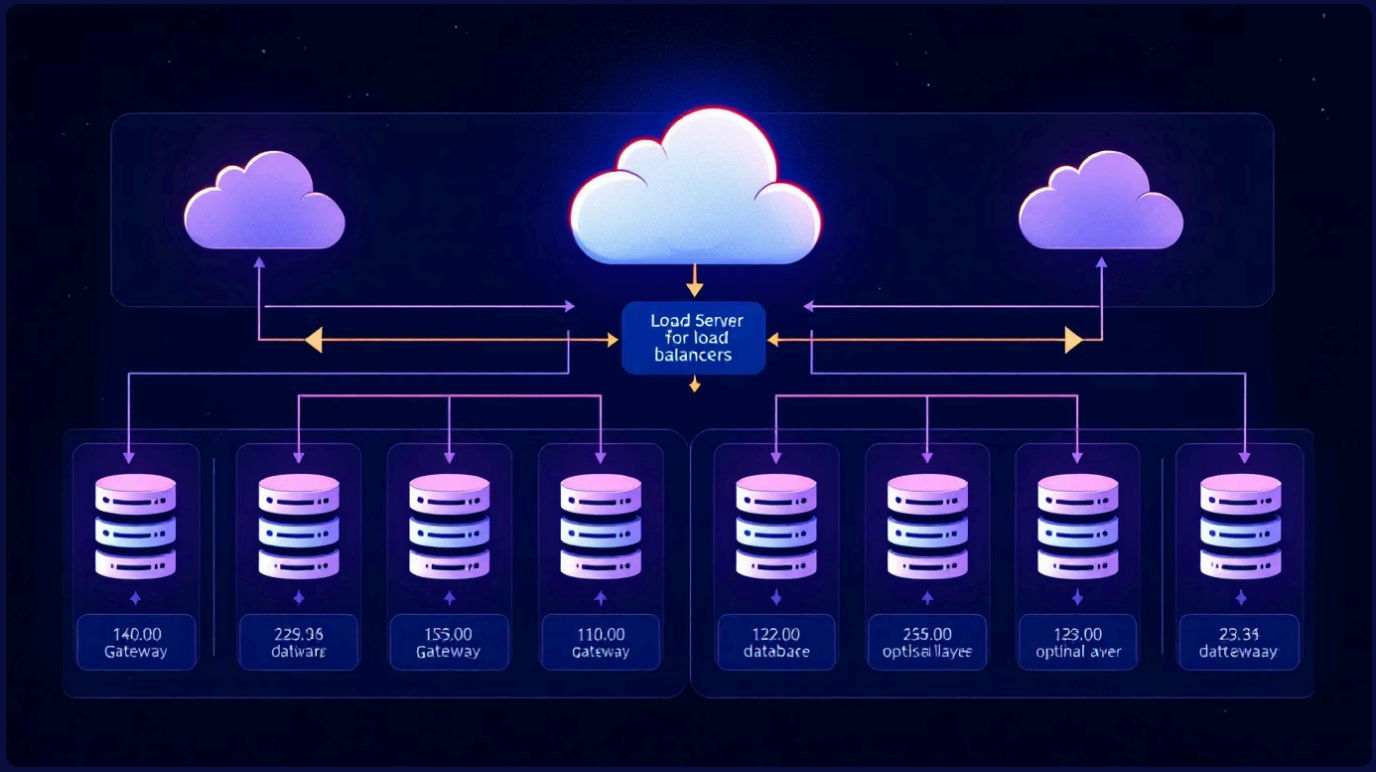
Made with GAMMA

Project Introduction

Modern cloud applications experience unpredictable traffic patterns that challenge traditional infrastructure approaches.

Managing infrastructure manually during traffic spikes is inefficient and introduces operational risk.

AWS Auto Scaling allows applications to automatically adjust the number of EC2 instances based on demand, ensuring high availability and efficient resource utilization.



This project demonstrates how Auto Scaling works together with an Application Load Balancer to create a scalable and resilient architecture.

Made with GAMMA

Problem with Static Infrastructure

Traditional infrastructure often uses a fixed number of servers, creating significant operational challenges:

Traffic Spikes

Cannot handle sudden increases in user demand without manual intervention

Manual Scaling

Requires engineers to constantly monitor and adjust capacity

Instance Failure

Single point of failure risks downtime and service disruption

Resource Waste

Over-provisioned servers lead to poor cost efficiency

Made with GAMMA

Project Objective

The objective of this project is to build a scalable and highly available architecture using AWS services.

Automatic Scaling

Automatically launch EC2 instances during traffic increase

Scale In

Automatically terminate instances during low demand

Self-Healing

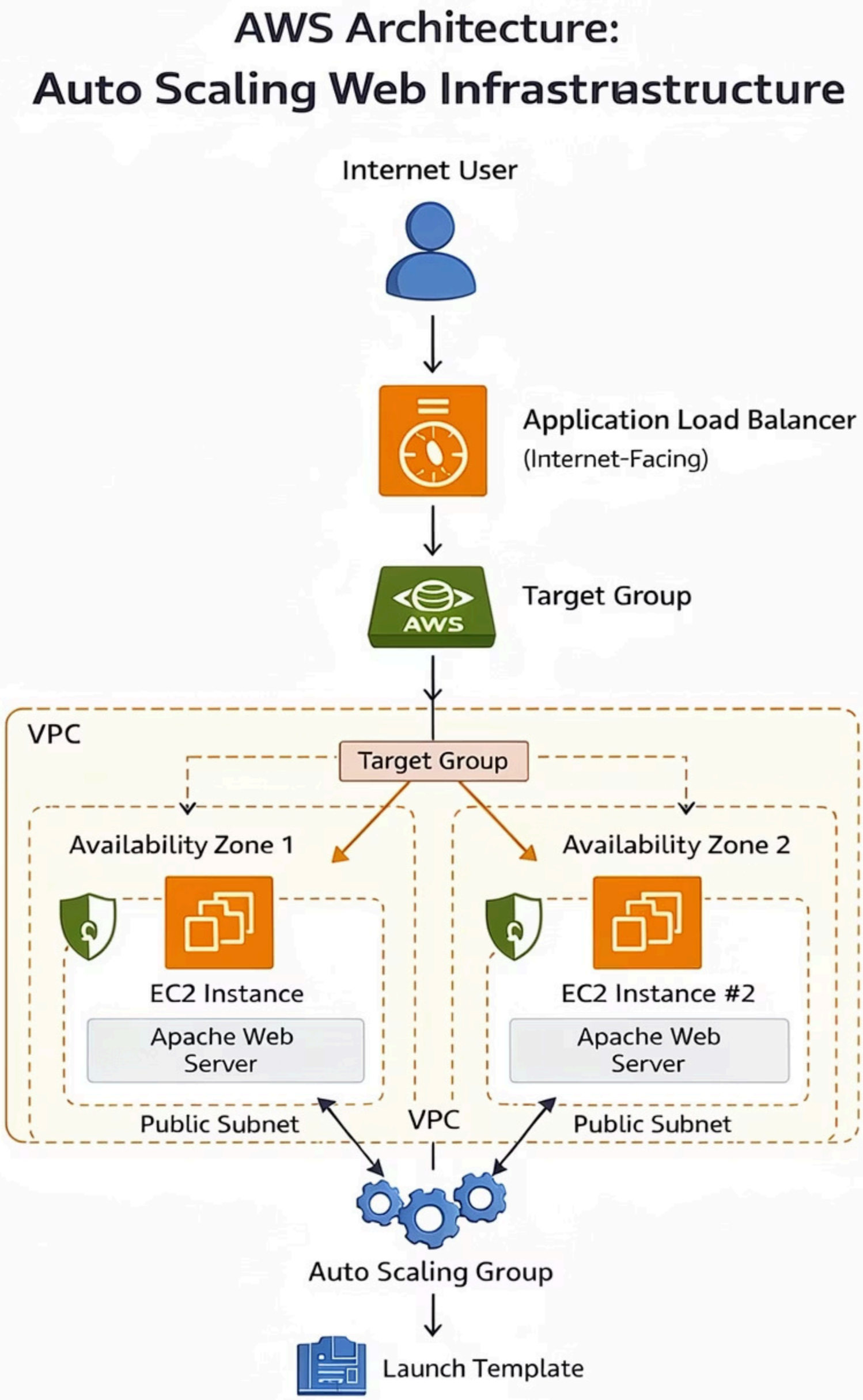
Automatically replace unhealthy instances

High Availability

Ensure application availability through load balancing

Made with GAMMA

Architecture Overview



This **architecture** demonstrates an **Auto Scaling web infrastructure** on AWS. Internet-facing Application Load Balancer routes traffic to a Target **Group** of EC2 instances in multiple Availability Zones managed by an **Auto Scaling Group.** The **Launch Template** defines the configuration for automatically launching new EC2 instareee **ilTemplate**.

Made with GAMMA

AWS Services Used

Amazon EC2

Virtual servers providing compute capacity

Launch Template

Blueprint defining EC2 instance configuration

Auto Scaling Group

Automatically manages EC2 capacity

Application Load Balancer

Distributes traffic across instances

Target Group

Logical grouping of EC2 instances

Amazon CloudWatch

Monitors system health and performance

Made with GAMMA

Implementation Details

Launch Template Configuration

The Launch Template acts as a blueprint for EC2 instances, defining:

- Amazon Machine Image (AMI)
- Instance type specification
- Security group configuration
- Key pair for authentication
- User data script for automation

Auto Scaling Group Settings

The Auto Scaling Group manages EC2 capacity automatically:

Desired Capacity: 2 instances

Minimum Instances: 1 instance

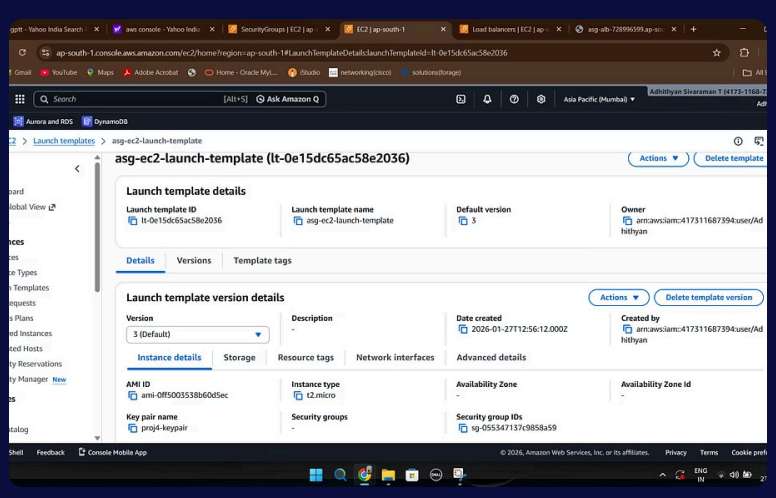
Maximum Instances: 2 instances

This ensures the application always maintains required compute capacity whilst remaining cost-efficient.

Made with GAMMA

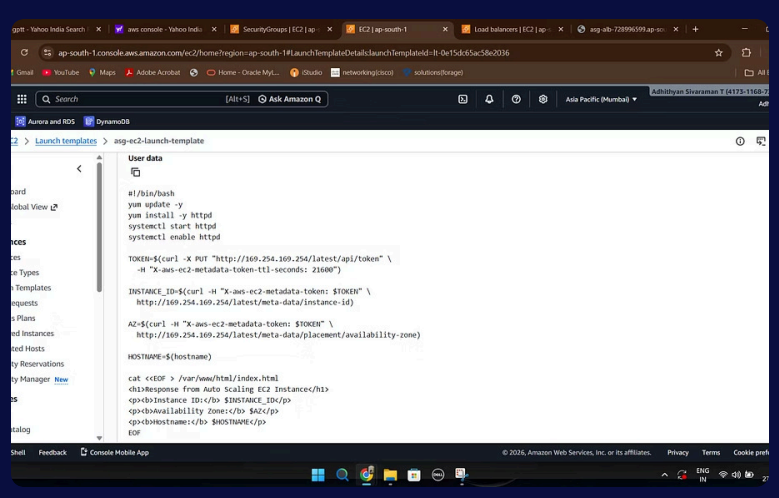
Implementation Screenshots

Screenshots from the deployment and configuration process:



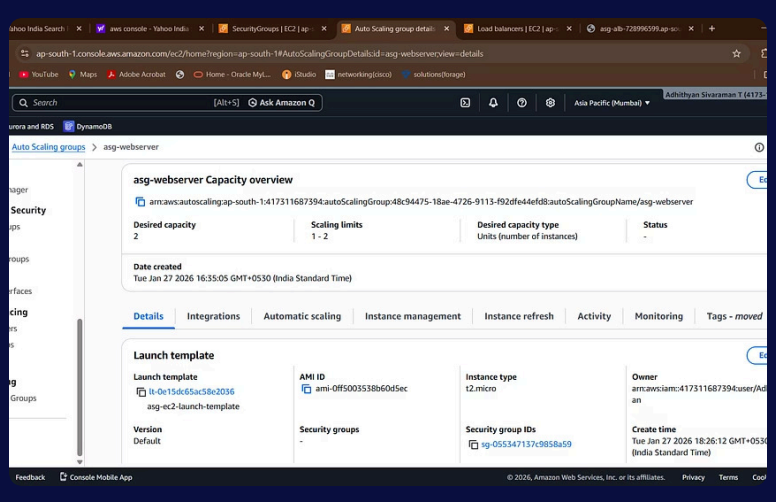
Launch Template Configuration

Launch Template created for the Auto Scaling Group including AMI ID, instance type, key pair, and security group configuration



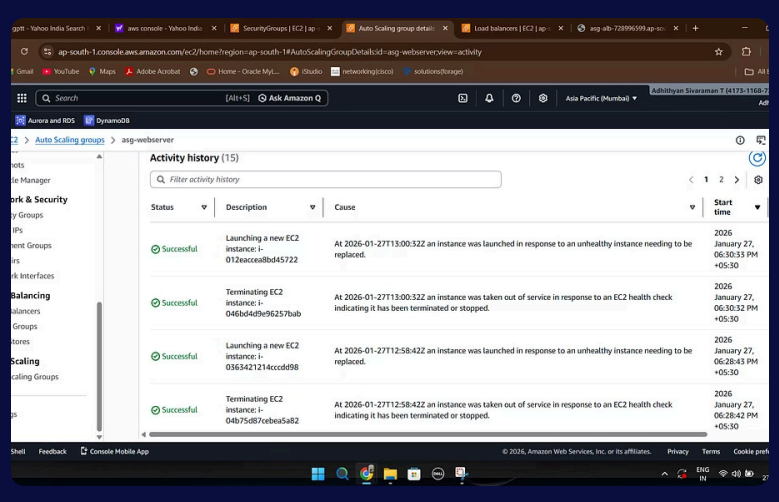
User Data Script

User data script used to automatically install and configure Apache web server on newly launched EC2 instances



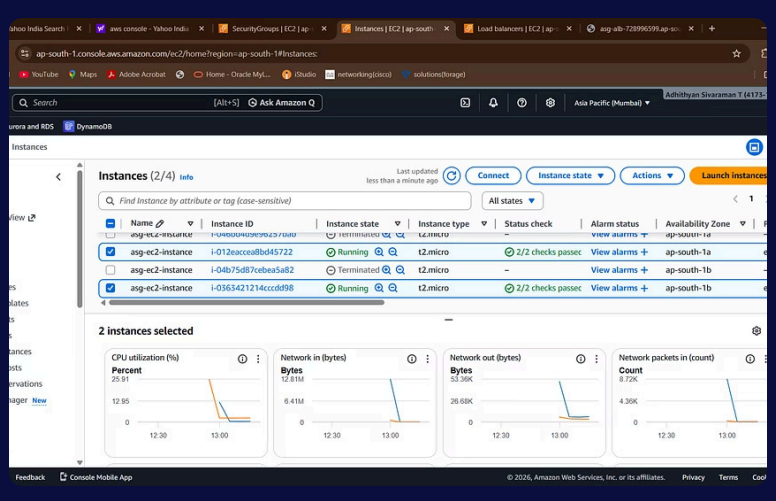
Auto Scaling Group Configuration

Auto Scaling Group settings including desired capacity and scaling limits



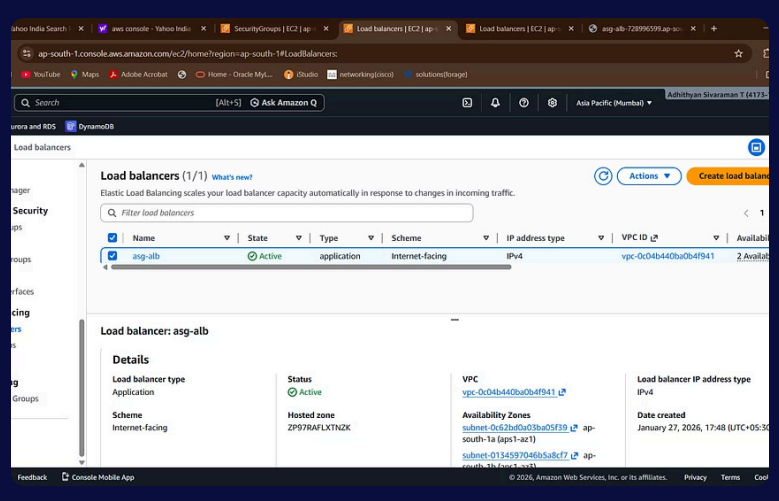
Auto Scaling Activity

Activity logs where AWS automatically terminates unhealthy instances and launches new instances



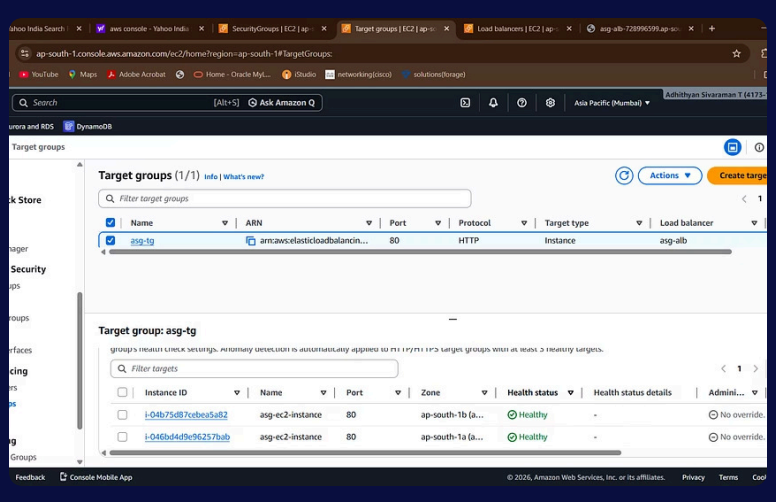
EC2 Instances Managed by ASG

EC2 instances created and managed automatically by the Auto Scaling Group



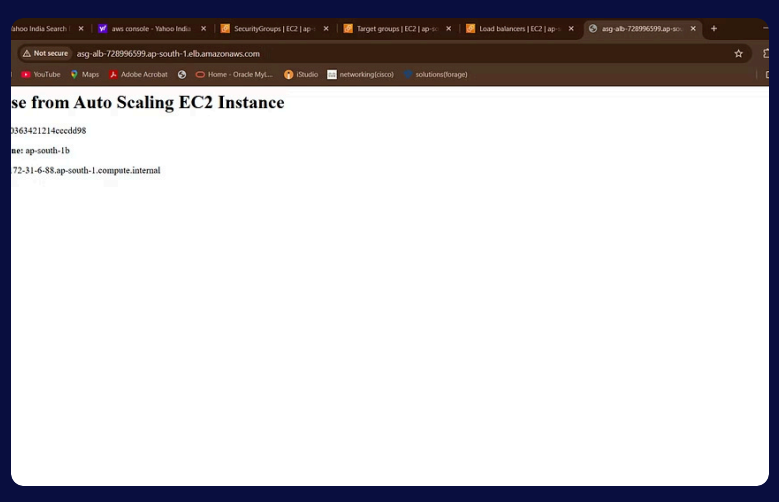
Application Load Balancer

Application Load Balancer configured to distribute traffic across EC2 instances



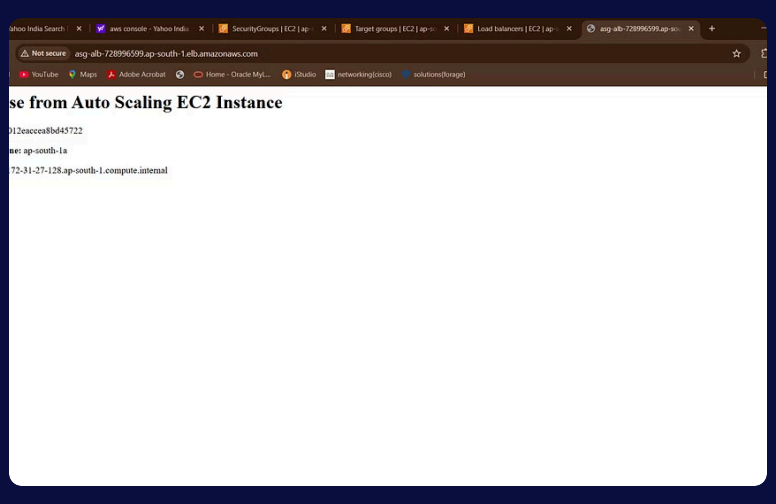
Target Group Health Check

Target Group where EC2 instances are registered and marked healthy



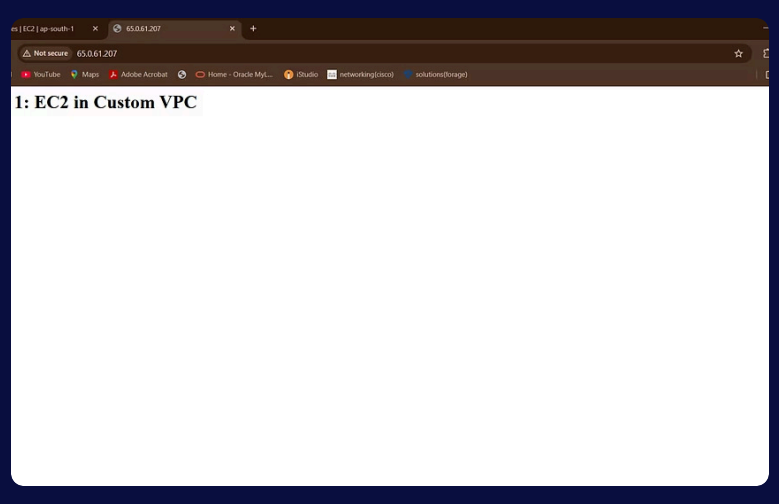
Load Balancer Output 1

Webpage response including instance ID and availability zone, demonstrating requests served by EC2 instances behind the load balancer



Load Balancer Output 2

Webpage response from a different instance behind the load balancer



Apache Webpage Output

Final rendered Apache test page served by the application stack

Made with GAMMA

Final Deployment Result

Load Balancer Integration

Application Load Balancer distributes incoming traffic across EC2 instances

Automated Scaling

Infrastructure automatically adapts to system demand

High Availability

Incoming traffic distributed across instances

The application is successfully deployed behind an Application Load Balancer and managed by an Auto Scaling Group.

Made with GAMMA

Conclusion and Key Takeaways

This project demonstrates how Auto Scaling and Application Load Balancer work together to create scalable and highly available cloud infrastructure.

The system can automatically respond to traffic changes, replace failed instances, and maintain application availability without manual intervention.

Auto Scaling Architecture

Load Balancing Concepts

High Availability Design

Automated Infrastructure Management

Production-Ready AWS Architecture

Made with GAMMA